

TASK AUTOMATION WITH PYTHON SCRIPTS

Submitted by:

Name: Saikam Bhavya Sri

College: Vignan's Nirula Institute of Technology and Science for Women

Department: Electronics and Communication Engineering

Aim:

To automate the process of fetching the title of a webpage, extracting it using regular expressions, saving it to a file, verifying file creation, and creating a backup copy.

Requirements:

- Python 3.x
- requests module
- Internet connection

Modules Used:

1. requests – To fetch webpage content.
2. re – To extract the webpage title.
3. os – To check whether the file exists.
4. shutil – To create a backup copy of the file.
5. File Handling – To create, read, and write files.

Algorithm:

1. Import the required modules.
2. Specify the webpage URL.
3. Fetch the webpage content using requests.
4. Extract the title using a regular expression.
5. Save the extracted title to a text file.
6. Check whether the file exists using `os.path.exists()`.
7. Create a backup copy using `shutil.copy()`.
8. Read and display the file contents.
9. Display the result.

Program:

```
# Task Automation Project
```

```
# Concepts Used: requests, re, file handling, os, shutil
```

```
import requests
```

```
import re
```

```
import os
```

```
import shutil
```

```
# Step 1: Fetch webpage content
```

```
url = "https://example.com"
```

```
response = requests.get(url)
```

```
if response.status_code == 200:
```

```
    html = response.text
```

```
# Step 2: Extract title using Regular Expression
```

```
match = re.search(r"<title>(.*?)</title>", html, re.IGNORECASE)
```

```
if match:
```

```
    title = match.group(1)
```

```
    # Step 3: Save title to a file
```

```
    with open("title.txt", "w") as file:
```

```
        file.write(title)
```

```
    print("Title Extracted:", title)
```

```
    # Step 4: Check file existence using os
```

```
    if os.path.exists("title.txt"):
```

```
        print("title.txt created successfully")
```

```
    # Step 5: Create backup using shutil
```

```
    shutil.copy("title.txt", "backup_title.txt")
```

```
    print("Backup file created successfully")
```

```
    # Step 6: Display file contents
```

```
    with open("title.txt", "r") as file:
```

```
        print("\nContent of title.txt:")
```

```
        print(file.read())
```

```
else:
```

```
    print("Title not found")
```

```
else:
```

```
    print("Failed to fetch webpage")
```

Output:

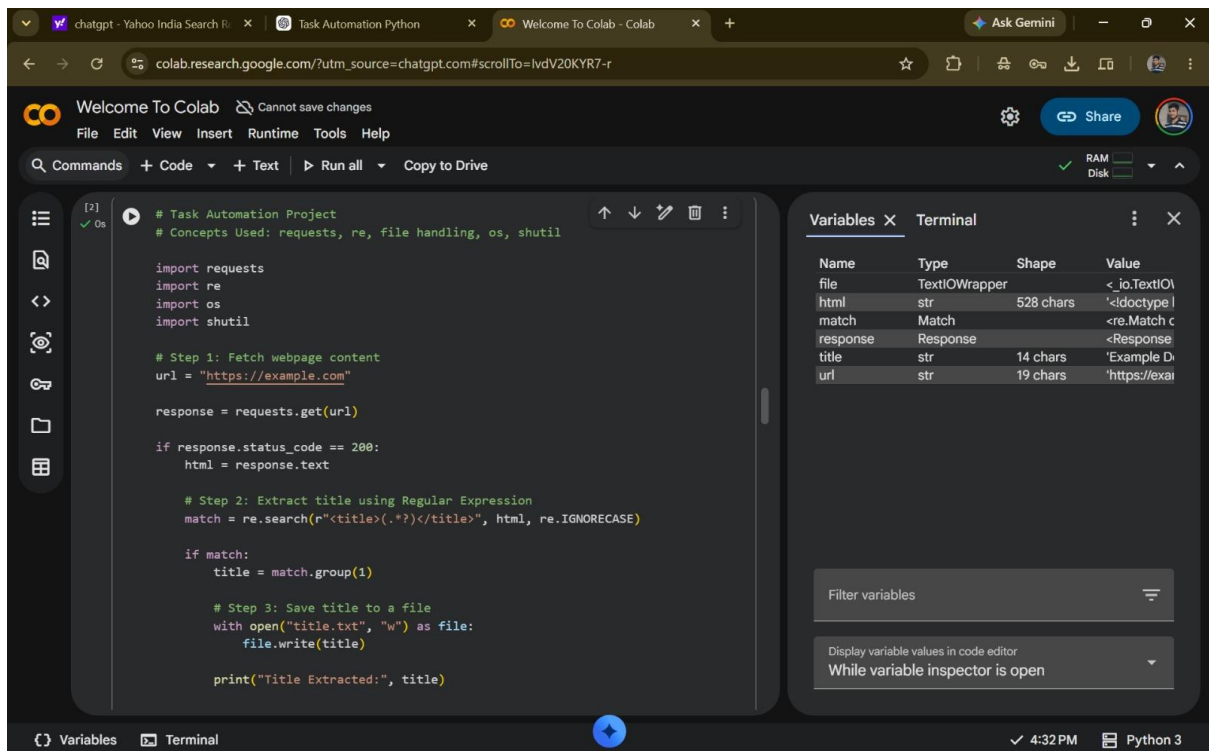
Title Extracted: Example Domain

title.txt created successfully

Backup file created successfully

Content of title.txt:

Example Domain



The screenshot shows the Google Colab interface with a Python script running. The script is titled "# Task Automation Project" and includes comments for concepts used (requests, re, file handling, os, shutil). It performs three steps: 1. Fetching webpage content from "https://example.com", 2. Extracting the title using a regular expression, and 3. Saving the title to a file named "title.txt". The output of the script is "Title Extracted:", followed by the title. The variable inspector on the right shows the state of variables: 'file' is a TextIOWrapper, 'html' is a string of 528 characters, 'match' is a Match object, 'response' is a Response object, 'title' is a string of 14 characters, and 'url' is a string of 19 characters. The bottom status bar indicates the time is 4:32 PM and the environment is Python 3.

```
[2] ✓ 0s # Task Automation Project
# Concepts Used: requests, re, file handling, os, shutil

import requests
import re
import os
import shutil

# Step 1: Fetch webpage content
url = "https://example.com"

response = requests.get(url)

if response.status_code == 200:
    html = response.text

# Step 2: Extract title using Regular Expression
match = re.search(r"<title>(.*?)</title>", html, re.IGNORECASE)

if match:
    title = match.group(1)

# Step 3: Save title to a file
with open("title.txt", "w") as file:
    file.write(title)

print("Title Extracted:", title)
```

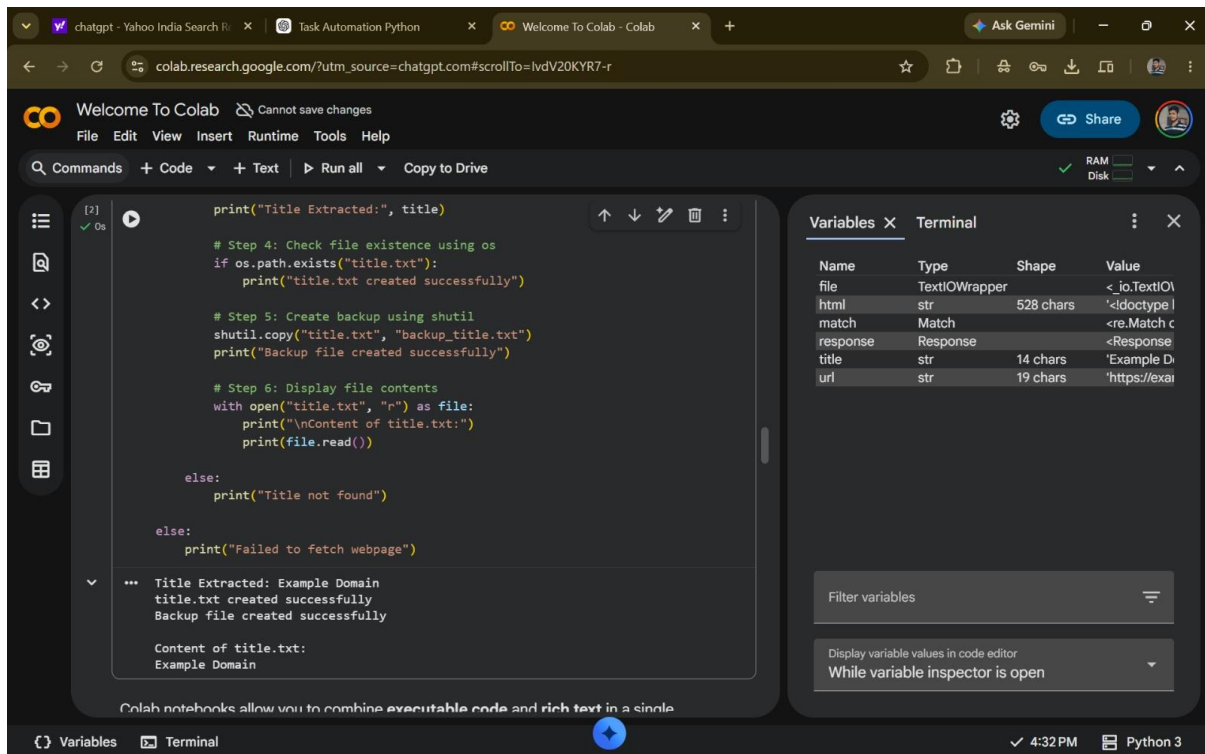
Name	Type	Shape	Value
file	TextIOWrapper		< io.TextIO
html	str	528 chars	'<doctype
match	Match		<re.Match c
response	Response		<Response
title	str	14 chars	'Example D
url	str	19 chars	'https://exa

Filter variables

Display variable values in code editor
While variable inspector is open

Variables Terminal

✓ 4:32 PM Python 3



Output Screenshots of the code while simulation.

Working

The program automates the task of extracting the title from a webpage and storing it for future use. First, the webpage content is fetched from a fixed URL using the Requests module. The downloaded HTML content is then analyzed using Regular Expressions (re) to identify and extract the text enclosed within the <title> tags.

After extracting the title, the program uses file handling operations to save the title into a text file named title.txt. The OS module is then used to verify whether the file has been successfully created. Once the file is confirmed, the Shutil module creates a backup copy of the file named backup_title.txt.

Finally, the saved title is read from the file and displayed to the user. This automation reduces manual effort in collecting webpage information and demonstrates the practical use of Python modules such as Requests, Regular Expressions, File Handling, OS, and Shutil.

Features

- Automatically fetches webpage content from the internet.
- Extracts webpage title using regular expressions.
- Stores extracted data in a text file.
- Verifies file creation using the OS module.
- Creates a backup copy of the file using Shutil.
- Reads and displays saved data.
- Reduces manual work through automation.
- Demonstrates multiple Python modules in a single application.

Applications

- Website monitoring and data collection.
- Content management systems.
- Automated report generation.
- Web scraping and information retrieval.
- Educational projects for learning Python automation.

Result:

The program successfully automated the extraction of a webpage title, saved it to a file, verified file creation, and created a backup copy using Python modules such as Requests, Regular Expressions, OS, Shutil, and File Handling.

Conclusion:

The Task Automation project was successfully implemented using Python. The program automatically fetched the title of a webpage, extracted the required information using regular expressions, and stored it in a text file. It also verified the file's existence using the OS module and created a backup copy using the Shutil module. The project demonstrated the effective use of **Requests, Regular Expressions, File Handling, OS, and Shutil** modules in automating a real-world repetitive task. Thus, the objective of automating webpage title extraction and storage was achieved successfully.